M1 IASD

HAI811I - Programmation Mobile

Rapport de Conception & Architecture - TP3

HALABLIAN Hugo

Faculté des Sciences
Université de Montpellier

Mars 2026

Sommaire

1 Introduction 3
2 Analyse des Besoins et Enjeux 3
  2.1 Objectifs Fonctionnels 3
  2.2 Contraintes Techniques 3
3 Choix de l'Architecture de Données : Pourquoi Room ? 4
4 Architecture de Persistance : Room Database 4
  4.1 Modélisation des Entités (Entities) 4
  4.2 Couche d'Accès aux Données (DAO) 4
5 Développement des Interfaces (UI/UX) 5
  5.1 Fragment d'Inscription et Validation 5
  5.2 Gestion du Calendrier 5
6 Mécanismes de Connexion et Sécurité 6
  6.1 Logique d'Authentification 6
7 Gestion du Planning Journalier 7
  7.1 Saisie et Persistance 7
  7.2 Synthèse Finale 7
8 Défis Techniques et Résolutions 8
  8.1 Problèmes de compilation Room 8
  8.2 Charte Graphique et Identité Visuelle 8
9 Conclusion 9
10 Annexes 10
  10.1 Code de la base de données 10
  10.2 Structure du projet 11

1 Introduction

L'évolution des applications mobiles modernes impose une gestion rigoureuse de la persistance des données locales. Ce TP s'inscrit dans cette démarche en exploitant la bibliothèque **Room**, une couche d'abstraction sur SQLite, pour structurer les données d'un utilisateur et son emploi du temps.

L'enjeu n'est pas seulement technique mais ergonomique : il s'agit de garantir l'intégrité des informations (validations) tout en offrant une navigation fluide entre inscription, authentification et gestion de planning.

2 Analyse des Besoins et Enjeux

2.1 Objectifs Fonctionnels

L'application doit répondre à trois piliers fonctionnels:

1. **La Persistance Identitaire** : Enregistrer un profil complet avec des métadonnées variées (nom, email, centres d'intérêt).
2. **La Sécurité Applicative** : Valider les formats de saisie (Regex) et vérifier l'unicité des comptes en base.
3. **La Gestion Temporelle** : Permettre la saisie d'un planning journalier découpé en créneaux horaires fixes.

2.2 Contraintes Techniques

Le passage de l'Exercice 1 à l'Exercice 2 a nécessité une refonte de la navigation, passant d'un flux linéaire « Inscription -> Affichage » à un hub de décision permettant la connexion d'utilisateurs existants.

Afin d'illustrer la complexité de l'application développée, le schéma suivant présente l'interaction entre les trois couches principales : l'interface utilisateur (UI), la logique métier (Validations) et la persistance (Room).

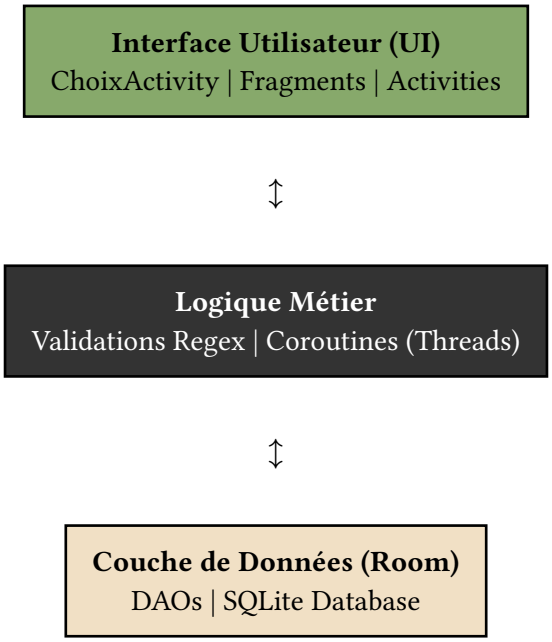

Fig. 1. – Représentation simplifiée de la hiérarchie logicielle

3 Choix de l'Architecture de Données : Pourquoi Room ?

Le sujet imposait l'utilisation d'une base de données[cite: 5, 13]. Bien qu'Android propose les **SharedPreferences** pour stocker des couples clé-valeur, ce mécanisme s'est avéré insuffisant pour les enjeux de ce TP :

- **Complexité structurelle** : Le planning journalier découpé en quatre créneaux nécessite une structure tabulaire que seul SQLite peut offrir de manière performante.
- **Requêtes relationnelles** : L'authentification lors de la connexion demande une recherche par login, opération optimisée par les index SQL de Room.
- **Évolutivité** : Contrairement au stockage plat, Room permet d'ajouter des colonnes ou de nouvelles tables (comme l'entité `Planning`) via des migrations gérées.

4 Architecture de Persistance : Room Database

4.1 Modélisation des Entités (Entities)

Nous avons défini deux entités distinctes pour respecter la forme normale de la base de données :

- **User** : Stocke les informations de profil. Le login sert de clé primaire, garantissant l'unicité technique demandée par le sujet.
- **Planning** : Lie chaque créneau horaire à un `userLogin`.

4.2 Couche d'Accès aux Données (DAO)

Les interfaces `UserDao` et `PlanningDao` utilisent des **Suspending Functions**. Cela permet d'exécuter les requêtes SQL sans bloquer le thread principal (UI Thread), évitant ainsi les erreurs de type « Application Not Responding » (ANR).

```
com.example.tp3_basededonnes

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query

@Dao
interface UserDao {
    // Permet d'insérer un utilisateur.
    // Si le login existe déjà, il sera remplacé[cite: 147, 156].
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertUser(user: User)

    // Permet de récupérer un utilisateur par son login pour la connexion[cite: 159].
    @Query("SELECT * FROM users_table WHERE login = :login LIMIT 1")
    suspend fun getUserByLogin(login: String): User?

    // Permet de récupérer tous les utilisateurs (utile pour les tests)
    @Query("SELECT * FROM users_table")
    suspend fun getAllUsers(): List<User>
}
```

Liste 1. – Implémentation du DAO avec requêtes de vérification d'existence

5 Développement des Interfaces (UI/UX)

5.1 Fragment d'Inscription et Validation

Le `FragmentInscription` est le point le plus complexe en termes de logique métier.

- **Regex de Login** : `^[a-zA-Z][a-zA-Z0-9]{0,9}$`. Elle assure que le login débute par une lettre et ne dépasse pas 10 caractères.
- **Longueur du Password** : Un contrôle strict à 6 caractères est effectué avant toute tentative d'insertion.

5.2 Gestion du Calendrier

Pour la date de naissance, l'utilisation d'un `DatePickerDialog` remplace la saisie manuelle. Cette approche garantit que la donnée stockée en base est toujours au format `JJ/MM/AAAA`, simplifiant les traitements futurs.

Fig. 2. – Interface d'inscription et sélecteur de date (DatePicker)

6 Mécanismes de Connexion et Sécurité

6.1 Logique d'Authentification

La `LoginActivity` interroge la base de données via le login saisi. Le processus est le suivant :

1. Recherche de l'objet `User` correspondant au login.
2. Si `null`, l'utilisateur n'existe pas.
3. Si non `null`, comparaison du mot de passe saisi avec celui de la base.
4. Redirection vers le planning en cas de succès, sinon affichage d'une erreur.

7 Gestion du Planning Journalier

7.1 Saisie et Persistance

L'activité `PlanningActivity` permet de remplir quatre champs correspondant aux créneaux demandés : 08h-10h, 10h-12h, 14h-16h et 16h-18h. L'enjeu ici est la mise à jour : l'utilisation de `OnConflictStrategy.REPLACE` dans le DAO permet à l'utilisateur de modifier son planning plusieurs fois par jour sans créer de doublons.

7.2 Synthèse Finale

Après validation, une activité de synthèse récupère le planning fraîchement enregistré pour confirmer la persistance.

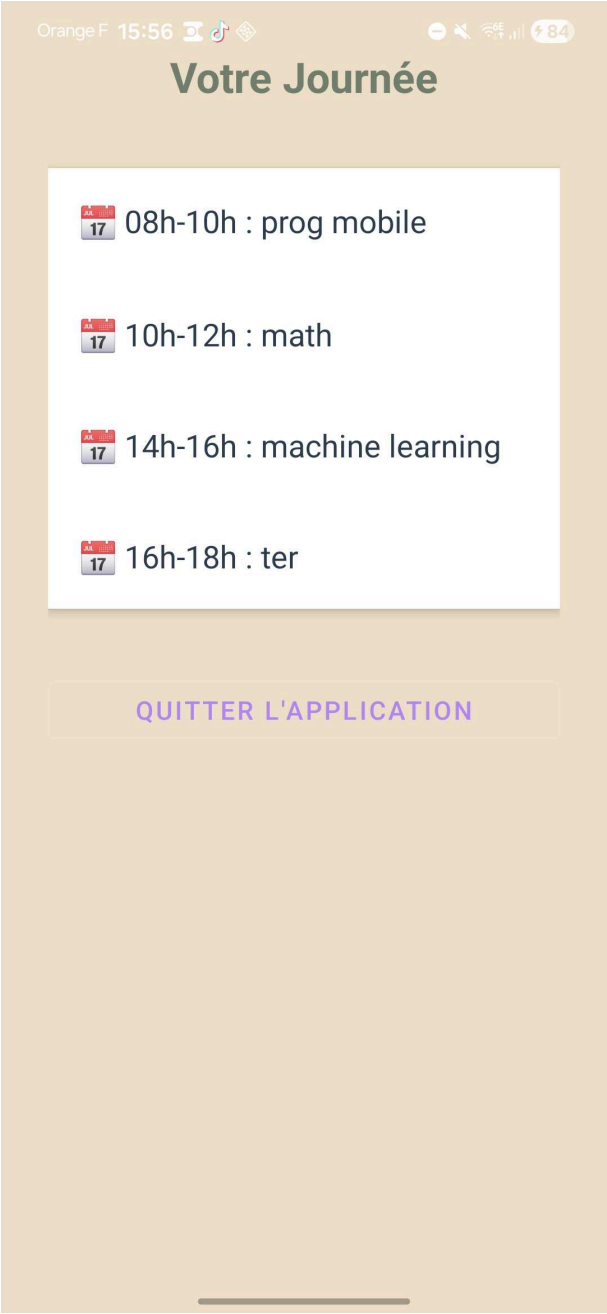

Fig. 3. – Vue de synthèse du planning journalier

8 Défis Techniques et Résolutions

8.1 Problèmes de compilation Room

Un défi majeur a été la configuration de **KAPT** avec le **JDK 21**. L'absence de génération automatique de la classe `AppDatabase_Impl` a initialement causé des `NullPointerException`.

- **Solution** : Alignement des `compilerOptions` sur le `JvmTarget` 17/21 et déclaration explicite des packages dans chaque fichier Kotlin.

8.2 Charte Graphique et Identité Visuelle

Pour répondre au besoin d'une interface « qualitative », nous avons opté pour :

- Une suppression de la `ActionBar` via le Manifest pour un rendu plein écran.
- Un espacement de 40px en haut de page pour éviter les collisions avec la barre d'état.
- Une palette basée sur le **Sage Green** (#87A96B) pour les éléments d'interaction.

Fig. 4. – Écran de choix Inscription/Connexion avec design personnalisé

9 Conclusion

Ce TP a permis d'appréhender la complexité de la gestion d'état dans une application Android multi-activités. L'utilisation de Room, couplée à une validation rigoureuse des entrées, assure une base solide pour toute évolution future (comme l'ajout d'un planning sur plusieurs jours ou la gestion de profils multiples).

10 Annexes

10.1 Code de la base de données

```
  package com.example.tp3_basededonnes
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [User::class, Planning::class], version = 2, exportSchema =
false) // On passe en version 2
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
    abstract fun planningDao(): PlanningDao // Ajout du DAO Planning

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: android.content.Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = androidx.room.Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "tp3_database"
                )
                    .fallbackToDestructiveMigration() // Important car on a changé la
version
                    .build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Liste 2. – Configuration de la base de données avec gestion des migrations

10.2 Structure du projet

```
app/src/main/java/com/example/tp3_basededonnes/
├── ChoixActivity.kt          (LAUNCHER - Point d'entrée)
├── MainActivity.kt           (Conteneur Inscription/Affichage)
├── LoginActivity.kt          (Authentification Exo 2)
├── PlanningActivity.kt       (Saisie Planning Exo 2)
├── SynthesePlanningActivity.kt (Affichage Final Exo 2)
│
├── database/                 (Package Base de Données)
│   ├── AppDatabase.kt        (RoomDatabase - Version 2)
│   ├── User.kt               (Entité Profil)
│   ├── UserDao.kt            (DAO Profil)
│   ├── Planning.kt           (Entité Planning Exo 2)
│   └── PlanningDao.kt        (DAO Planning Exo 2)
│
└── fragments/                (Package UI Fragments)
    ├── FragmentInscription.kt (Logic Inscription & Regex)
    └── FragmentAffichage.kt   (Synthèse Inscription)
```

Comme illustré dans l'arborescence ci-dessus, l'architecture du projet a été restructurée pour répondre aux exigences de l'Exercice 2. Nous avons opté pour une organisation par packages fonctionnels :

- **`database/`** : Regroupe l'intégralité de la couche de persistance Room. L'ajout de `Planning.kt` et `PlanningDao.kt` a nécessité une migration de la base de données vers la **version 2**, gérée dans `AppDatabase.kt`.
- **`fragments/`** : Contient la logique d'interface utilisateur (UI) pour le flux d'inscription initiale, permettant de découpler la présentation de la logique métier.
- **Activités principales** : `ChoixActivity` agit comme le nouveau routeur de l'application, aiguillant l'utilisateur soit vers le flux d'inscription (`MainActivity`), soit vers le flux de connexion (`LoginActivity`), conformément au workflow de l'Exercice 2.